CiV Dossier Engine

Architecture & Build Plan - n8n + Supabase + Kimi K2 / DeepSeek V3

Version	1.0
Date	2026-08-20
Owner	Rad - Technical Operations, Compliance IV
Requested by	Stu - Sales Lead
Scope	504 prospects x 134 datapoints, one dossier per company
Stack	n8n (Hostinger) - Supabase hoyinfxpotpwkhzsgime - OpenRouter
Structure	Part I architecture + Appendix A implementation plan

Contents

Part I — Architecture

1. Document control

Field	Value
Document	CiV Dossier Engine — Architecture & Build Plan
Version	1.0
Date	2026-08-20
Owner	Rad — Technical Operations, Compliance IV
Requested by	Stu — Sales Lead, Compliance IV
Intended readers	AI build agents executing Appendix A; Rad reviewing gate evidence
Companion artifact	`plans/civ-dossier-engine-implementation.md` (embedded as Appendix A)
Target database	Supabase project `hoyinfxpotpwkhzsgime` (504 companies loaded)
Target orchestrator	n8n, self-hosted on Hostinger (105 workflows present)
Runtime models	Kimi K2 and DeepSeek V3 via OpenRouter credential `WkivISO0KPmV0dNq`
Catalog	134 publicly-collectible datapoints across 18 categories
Chunk graph	30 chunks already defined in `civ_chunk_def`

How to use this document. Read Part I once for orientation — it explains why the system is shaped the way it is and which invariants must not be broken. Then execute strictly from Appendix A, which carries the complete Data Definition Language (DDL), the workflow node lists, the code, and the verification query for every task. Where the two disagree, Appendix A is authoritative for *what to type* and Part I is authoritative for *what must remain true*.

Acronyms are spelled out at first use throughout: TCPA (Telephone Consumer Protection Act), DNC (Do Not Call), PEWC (Prior Express Written Consent), DDL (Data Definition Language), API (Application Programming Interface), LLM (Large Language Model), ICP (Ideal Customer Profile), TTL (Time To Live), DOM (Document Object Model), BPO (Business Process Outsourcing), DID (Direct Inward Dialing number), NANPA (North American Numbering Plan Administrator), FCC (Federal Communications Commission), FTC (Federal Trade Commission), BBB (Better Business Bureau).

2. Executive summary

The CiV Dossier Engine collects 134 publicly-verifiable datapoints about each of 504 prospect companies and produces one evidence-cited research dossier per company, unattended. It runs as five n8n workflows over a Supabase database, dispatching work through a 30-node dependency graph in which each node — a "chunk" — collects between three and fifteen related datapoints and nothing else. No agent ever sees the whole company. The correctness guarantee that makes the output usable is structural rather than procedural: every stored value must reference a stored artifact and quote a verbatim string from it, enforced by a database CHECK constraint, and a release gate refuses to render any dossier whose quotes cannot be found in the bytes on file. A human reads the dossier and decides what to send; the engine writes no outbound message.

The build is segmented into five phases and nineteen work packages. Phase A establishes the findings table, the claimable run queue and the dependency-resolution view. Phase B loads the catalog, adds an n8n execution layer to the existing chunk graph, writes the anti-fabrication prompts and seeds the state-statute overlay. Phase C builds the three orchestration workflows — budget guard, scheduler and chunk runner. Phase D implements the chunk bodies across five archetypes, of which eight run as pure SQL with no model in the path. Phase E adds scoring, the citation gate, dossier composition, the refresh scheduler and a twenty-company pilot. Every work package carries an acceptance gate stated as a query whose expected output is written down, so a reviewer can

demand literal evidence rather than a status report.

3. Architecture principles

P1 — Chunk by dependency, never by convenience. Work is dispatched only from `civ_ready_chunk`, a view that derives runnable work from the `depends_on` arrays already stored in `civ_chunk_def`. *Rationale:* the chunk order is data, so regrouping datapoints or inserting a chunk is a SQL `UPDATE`, never a workflow edit. n8n contains no knowledge of the graph.

P2 — A value without evidence cannot be stored. `civ_finding.citation_gate` is a CHECK constraint requiring `artifact_id`, `source_url` and an `evidence_quote` of at least twelve characters for every non-null value. *Rationale:* models asked for an unobtainable fact invent one. Prompt instructions reduce this; a constraint eliminates it. Verified in WP A1 by an insert that must fail.

P3 — Quotes are verified against stored bytes, not against the live web. `civ_artifact` holds the body every quote came from, so `civ_citation_violation` checks 100% of findings with a SQL substring test and no HTTP request. *Rationale:* sampling misses systematic prompt failures, and re-fetching introduces a second source of truth that drifts.

P4 — Single writer for every rollup. `companies.civ` is written only by `civ_project_company()`, called once per company by PC-23. *Rationale:* twenty-four per-company chunks running concurrently against one JSONB column is a lost-update race in which the last writer silently discards the rest.

P5 — No model where a join will do. Eight of the thirty chunks are declared `archetype = 'sql'` and carry no model. *Rationale:* placing an LLM in front of a REST endpoint that returns clean JSON, or in front of a Postgres join, adds cost, latency and a fabrication surface for no gain. Verified in WP D2 by a query asserting zero recorded models on those chunks.

P6 — Unreachable datapoints are declared, not discovered. The 25 datapoints marked `Partial` in the catalog carry `method = 'ask'` and are filtered out of every agent dispatch. *Rationale:* an agent that flails and then concludes unreachability is in exactly the state where it fabricates instead. Their `discovery_question` renders in the dossier as a question for the sales call.

P7 — Cost and concurrency are bounded by the database, not by hope. `civ_budget` halts every workflow on a daily cap; `civ_claim_chunks` bounds in-flight work; `max_tool_calls` bounds each agent. *Rationale:* an unattended loop against paid APIs with no cap is how a weekend produces a four-figure bill.

P8 — Allegations are never described as findings. PC-22 rejects any composed text that states a legal conclusion. *Rationale:* CiV's positioning is compliance risk management, never legal advice, and a dossier that reads as an accusation is unusable in the sales conversation it exists to support.

4. System context

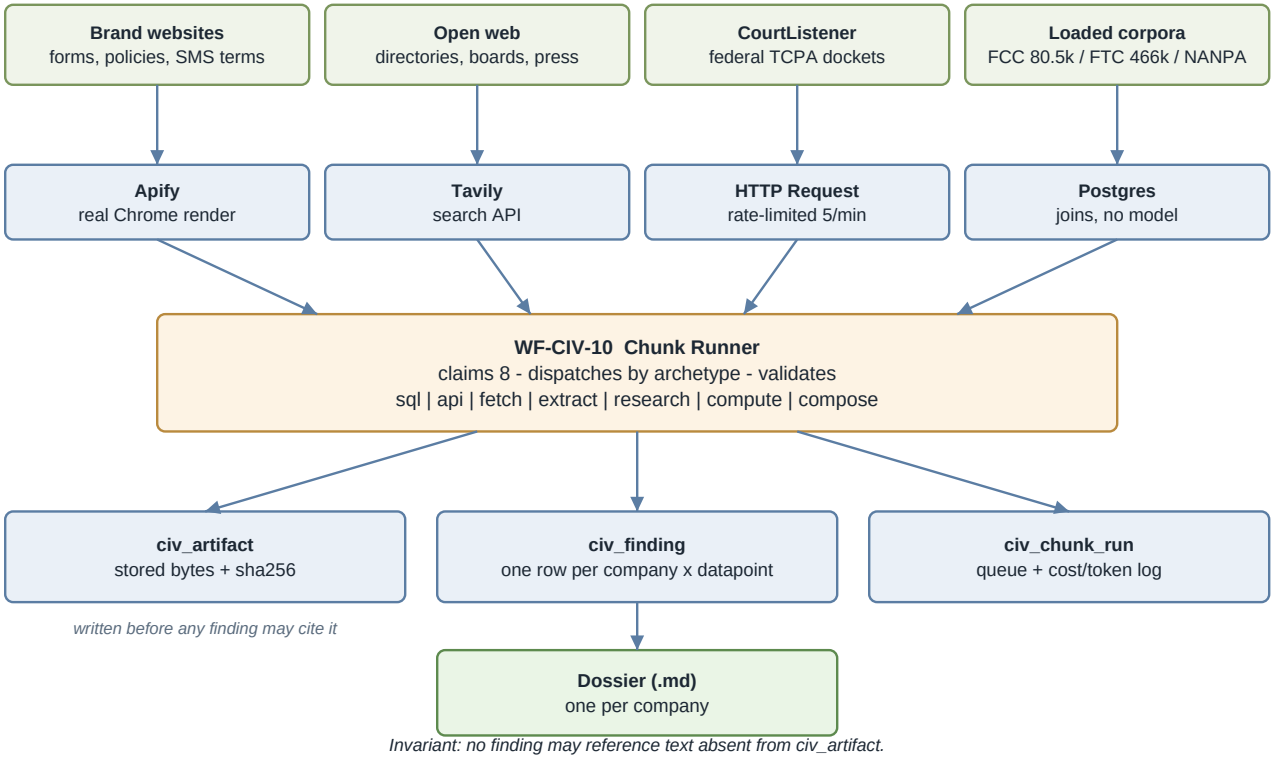


Figure 1 - System context: every external source deposits bytes into civ_artifact before any finding may cite it.

External source	Entry point	Identity key	Returns into
Company brand websites	Apify actor (real Chrome)	brand domain	`civ_artifact` (kind `form_probe`, `policy`, `page`)
Open web / directories	Tavily search API	query string	`civ_artifact` (kind `page`)
CourtListener	REST `/api/rest/v4/search/?type=r&suitNature=485`	docket id	`court_cases`, `civ_artifact`
FCC complaint corpus	already loaded — `fcc_violations` (80,536 rows)	telephone number	`civ_finding` M01–M05
FTC DNC corpus	already loaded — `ftc_dnc` (466,196 rows)	telephone number	`civ_finding` M01–M05
NANPA block assignments	already loaded — `npanxx_blocks` (204,462 rows)	NPA-NXX	`civ_finding` G05–G09
State statutes	REF-02, primary-source text only	state code	`civ_state_overlay`
LinkedIn	Airscale credential `vRVJcn1nRmJtDkLw`	company URL	`civ_artifact`
Slack `#civ-ops`, `#civ-dossiers`	outbound only	—	operator notifications

The mechanism that unifies these is `civ_artifact`. Every source, whether an agent read it or a deterministic fetch retrieved it, deposits its bytes there with a SHA-256 hash before any finding may reference it. That single rule is what lets a quote in a dossier be checked without leaving the database, and it is why the citation audit costs nothing to run at full coverage.

5. Data architecture

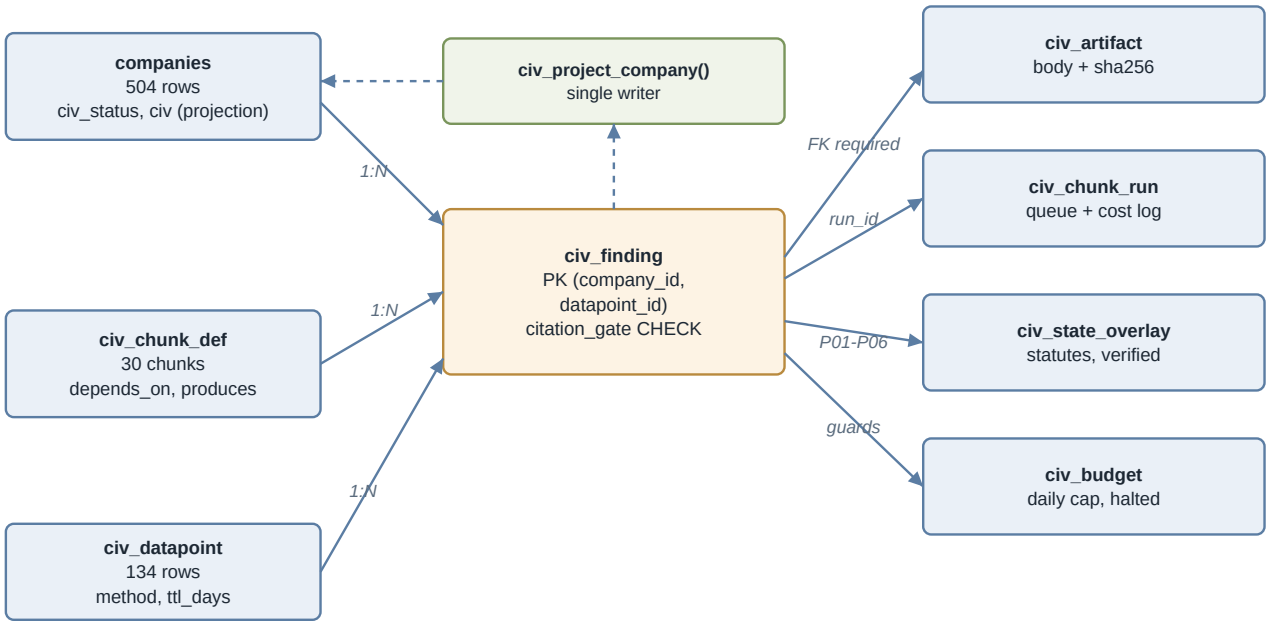


Figure 2 - Entity model: civ_finding is the hub; the citation_gate CHECK makes an uncited value unstorable.

Table	Role	Key columns
`companies`	504 prospects; pipeline state	`id`, `civ_status`, `civ` (projection), `website`
`civ_chunk_def`	The 30-chunk graph + execution config	`chunk_id`, `depends_on`, `produces`, `archetype`, `model`
`civ_datapoint`	The 134-datapoint catalog	`datapoint_id`, `chunk_id`, `method`, `ttl_days`, `discovery_question`
`civ_chunk_run`	Claimable work queue and run log	`run_id`, `company_id`, `chunk_id`, `status`, `cost_usd`
`civ_artifact`	Stored bytes behind every quote	`artifact_id`, `company_id`, `url`, `body`, `sha256`
`civ_finding`	One row per company × datapoint	PK `(company_id, datapoint_id)`, `evidence_quote`, `expires_at`
`civ_state_overlay`	Primary-source statutory text	`state`, `citation`, `verification_string`, `verified`
`civ_budget`	Daily spend cap and halt flag	`day`, `llm_cost_usd`, `halted`
`vendors`, `vendor_sellers`	Vendor litigation graph (14 / 35 rows)	`vendor_name`, `case_count`, `settled_count`

Identity strategy. `companies.id` is the uuid every table hangs from. Datapoint identifiers are the category letter plus a zero-padded ordinal (`A01`, `D09`, `S01`) and already appear in `civ_chunk_def.produces`, so WP B1 binds catalog to graph with a single join rather than a manual mapping. Chunk identifiers are the existing `PC-nn`, `REF-nn`, `SCR-nn` strings.

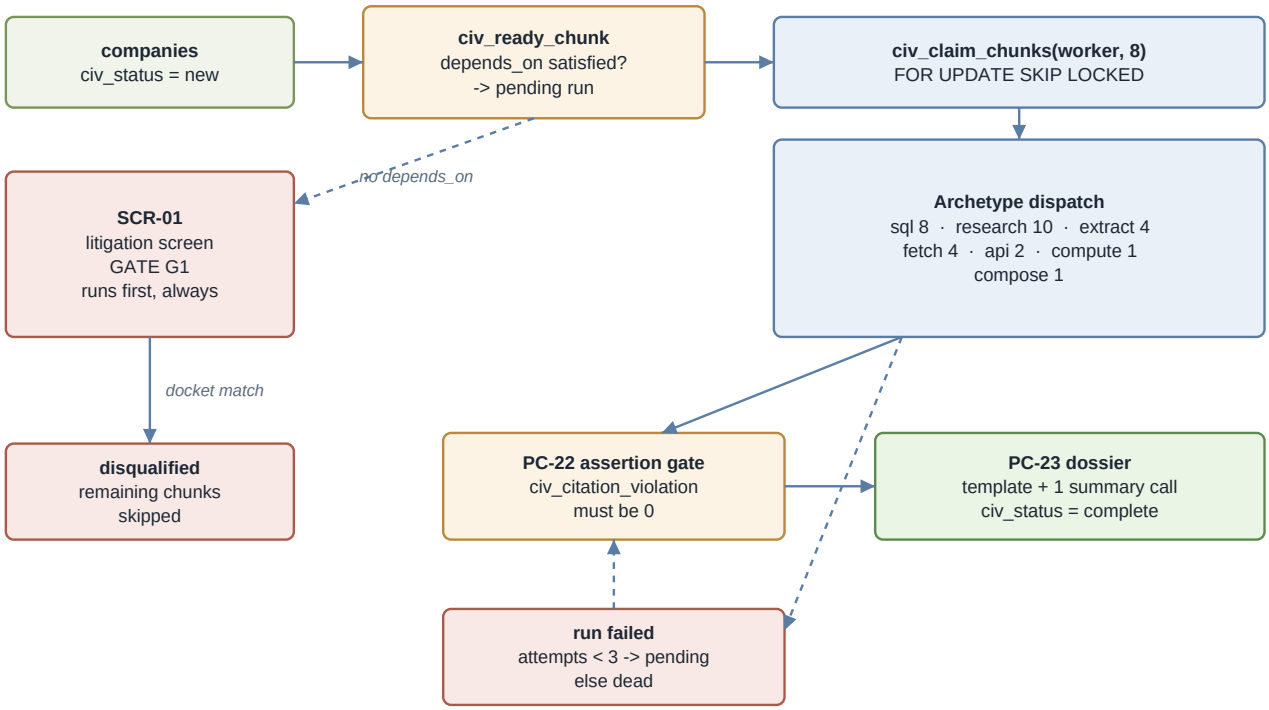
Single-writer rule. `civ_finding` is written by many chunks but each row belongs to exactly one chunk, because `civ_datapoint.chunk_id` assigns every datapoint to one producer. The primary key `(company_id, datapoint_id)` therefore cannot be contended by two concurrent chunks. `companies.civ` has one writer, `civ_project_company()` (P4).

Interface catalog.

Interface	Purpose	Called by
`civ_ready_chunk` (view)	Which company × chunk pairs are runnable now	WF-CIV-01 scheduler
`civ_claim_chunks(worker, limit)`	Atomically claim N pending runs	WF-CIV-10 runner
`civ_citation_violation` (view)	Findings whose quote is absent from its artifact	PC-22 gate, WF-CIV-05 audit
`civ_project_company(uuid)`	Collapse findings into `companies.civ`	PC-23 only
`classify_tns` (existing RPC)	Line type and carrier per number	PC-11

Full DDL is in Appendix A Tasks A1–A4; the catalog loader is Task B1.

6. Behavior architecture



Caps that terminate every loop: max_tool_calls 12, max_attempts 3, 30-min claim reaper, daily civ_budget halt.

Figure 3 - Behavior: dependency-driven dispatch, SCR-01 as the disqualifier gate, PC-22 as the release gate.

Seven archetypes, assigned in Task B2, determine how a claimed chunk executes:

Archetype	Chunks	Model	Tools	What it writes
`sql`	8	none	none	findings from joins
`api`	2	DeepSeek V3	HTTP, Postgres	artifacts + findings
`fetch`	4	none	Apify render/scrape	artifacts only
`extract`	4	DeepSeek V3	none	findings from stored artifacts
`research`	10	Kimi K2	Tavily, Apify	artifacts + findings
`compute`	1	none	none	derived scores
`compose`	1	DeepSeek V3	none	dossier file

Routing rules.

1. A company enters at `civ_status = 'new'` and is admitted to `'queued'` by the scheduler, which holds the working set at 40 concurrent companies.
2. `civ_ready_chunk` emits a chunk only when every entry in its `depends_on` array has a run with status `done` for that company. SCR-01 is the only per-company chunk with no dependencies, so it is always first.
3. SCR-01 is the disqualifier gate. A confirmed federal TCPA docket sets `civ_status = 'disqualified'` and marks the company's remaining chunks `skipped`. CiV is prevention-only; the catalog annotates datapoint N02 as *"ICP disqualifier"*.
4. Rate-limited chunks (`SCR-01`, `REF-05`) receive priority 10 and are claimed ahead of others.
5. A failed run returns to `pending` with `last_error` set; after three attempts the guard marks it `dead`, which releases dependent chunks from ever becoming ready — the company ends `failed` rather than silently incomplete.
6. PC-22 is the release gate: any row in `civ_citation_violation` for that company blocks PC-23.
7. PC-23 composes, calls `civ_project_company()`, and sets `civ_status = 'complete'`.

Loop-termination guarantees. Agent loops terminate on `max_tool_calls` (default 12, per chunk). Run retries terminate on `max_attempts` (3). Stuck claims are reaped after 30 minutes. The daily spend cap in `civ_budget` halts all five workflows regardless of queue depth. There is no unbounded loop anywhere in the design.

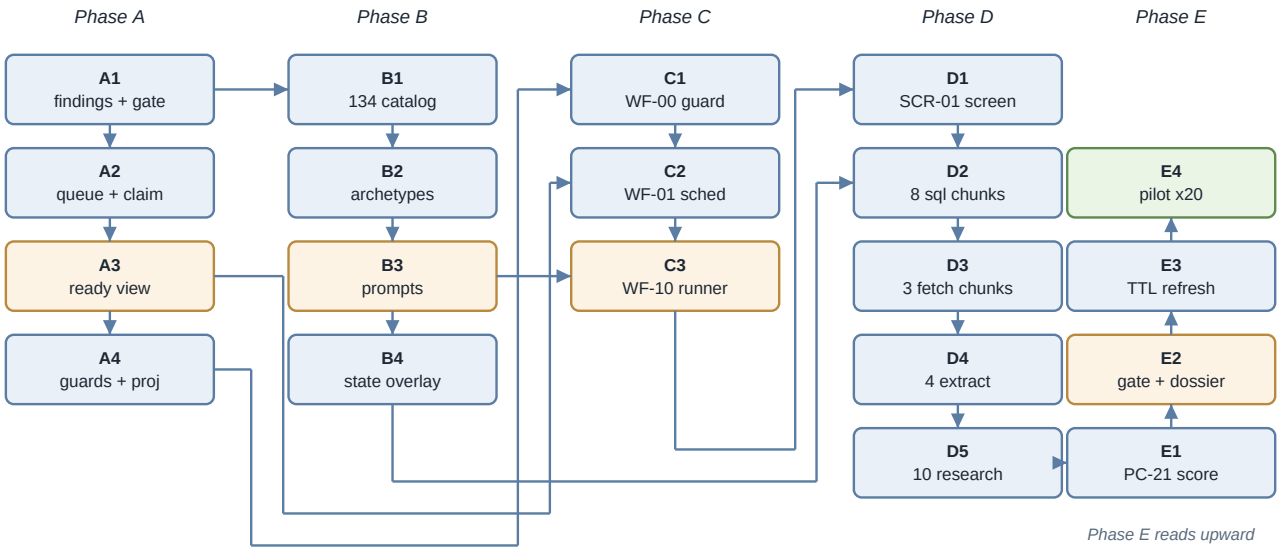
Failure policy. Failures are recorded, never silently swallowed: `civ_chunk_run.last_error` holds the message, `#civ-ops` receives the budget halt, and a `dead` chunk leaves the company diagnosable through `civ_chunks` in the projection.

7. Workflow catalog

ID	Workflow	Trigger	Responsibility	Depends on
WF-CIV-00	Guard and Reaper	`*/5 * * * *`	Roll up spend, set halt flag, reap stuck claims, kill exhausted runs	A2, A4
WF-CIV-01	Scheduler	`*/15 * * * *`	Admit companies, enqueue ready chunks from the view	A3, C1
WF-CIV-10	Chunk Runner	`*/5 * * * *`	Claim, dispatch by archetype, validate, write findings	C2, B2, B3
WF-CIV-20	Refresh Scheduler	`0 3 * * *`	Re-enqueue chunks whose findings passed their TTL	E2
WF-CIV-05	Citation Audit	`0 2 * * *`	Report `civ_citation_violation` counts per chunk to Slack	A4

Build order follows the dependency column: WF-CIV-00 first because every other workflow reads its halt flag, then the scheduler, then the runner, then the two nightly jobs. The existing *NANPA Monthly Refresh* workflow (`LX7srZ3KlMeQVmYy`) already implements REF-03 and is called rather than rebuilt. Per-node specifications are in Appendix A Tasks C1–C3 and E3.

8. Work breakdown structure



Solid arrows = build dependency; they match the Depends-on column of section 8.

Critical path A1 > A2 > A3 > C2 > C3 > D3 > D4 > E1 > E2 > E4.

Phase B runs parallel to A after A1; D2, D3 and D5 are mutually independent after C3.

Figure 4 - Work-package dependency graph (19 packages across 5 phases).

Phase A — Foundation

WP	Task	Scope	Depends on	Gate	Model
A1	A1	`civ_datapoint` + `civ_finding` with `citation_gate` CHECK	—	Insert of an uncited value raises `violates check constraint "citation_gate"`	Sonnet
A2	A2	`civ_chunk_run` queue + `civ_claim_chunks` with SKIP LOCKED	A1	3 rows inserted, `civ_claim_chunks(w,2)` returns exactly 2; statuses read claimed=2 pending=1	Sonnet
A3	A3	`civ_ready_chunk` dependency view	A2	With one company queued, view returns exactly one row and it is `SCR-01`	Opus
A4	A4	`civ_budget`, `civ_citation_violation`, `civ_project_company()`	A1, A2	`civ_citation_violation` returns 0; `civ_project_company()` returns `{}`; budget row reads halted=false cap=20.00	Sonnet

Phase B — Definitions (parallel with A after A1)

WP	Task	Scope	Depends on	Gate	Model
B1	B1	Load 134-row catalog, bind to graph, derive discovery questions	A1	`civ_datapoint` count = 134; null `chunk_id` = 0; `discovery_question` not null = 25; per-category counts match A 12 / D 15 / N 10	Sonnet
B2	B2	Archetype, model, tool allowlist columns on `civ_chunk_def`	—	Archetype counts read api 2, compose 1, compute 1, extract 4, fetch 4, research 10, sql 8; null archetype = 0	Sonnet
B3	B3	Anti-fabrication preamble + per-chunk task prompts	B2	Every `extract`/`research`/`api`/`compose` chunk has non-null `system_prompt`; PC-07/08/09 exceed the shared preamble length	Opus
B4	B4	Seed `civ_state_overlay` for mini-TCPA footprint states	—	>=12 rows, all with `citation`, `canonical_url`, `verification_string`; >=12 `verified` after REF-01	Opus

Phase C — Core orchestration

WP	Task	Scope	Depends on	Gate	Model
C1	C1	WF-CIV-00 guard and reaper	A2, A4	Cap set to 0 -> `halted` reads true and one Slack message lands in `#civ-ops`; cap restored	Sonnet
C2	C2	WF-CIV-01 scheduler	A3, C1	After one manual execution: `civ_chunk_run` = 40 rows, `select distinct chunk_id` returns only `SCR-01`	Opus
C3	C3	WF-CIV-10 runner, seven-archetype dispatch	C2, B2, B3	One company through SCR-01: run status `done`, N01–N10 findings present, `civ_citation_violation` = 0	Opus

Phase D — Chunk implementations (parallel after C3)

WP	Task	Scope	Depends on	Gate	Model
D1	D1	SCR-01 litigation disqualifier with name adjudication	C3	Disqualified share falls between 20% and 40%; three matched dockets manually confirmed as the same company	Opus
D2	D2	Eight SQL chunk bodies	C3, B4	Each chunk has `done` runs; `select model from civ_chunk_run where chunk_id in (...) and model is not null` returns 0 rows	Sonnet
D3	D3	PC-05/PC-06/PC-10 acquisition, real Chrome for the form probe	C3	`civ_artifact` null `sha256` = 0; a `form_probe` artifact contains a non-empty `checkboxes` array with `defaultChecked`	Opus
D4	D4	PC-07/PC-08/PC-09/PC-14 extraction	D3	`civ_citation_violation` = 0; 9 of 10 hand-checked quotes located at their `source_url`	Opus
D5	D5	Eight agentic research chunks on Kimi K2	C3	`civ_citation_violation` = 0; no chunk averaging its `max_tool_calls` ceiling	Opus

Phase E — Scoring, gate, dossier, pilot

WP	Task	Scope	Depends on	Gate	Model
E1	E1	PC-21 exposure scoring from `scoring_weights`	D2, D4, D5	Every S% finding carries a non-empty `derived_from` array; count of null `derived_from` = 0	Sonnet
E2	E2	PC-22 citation gate and PC-23 dossier composition	E1	`civ_citation_violation` = 0; three dossiers read cold and Stu confirms he can write an opener from each	Opus
E3	E3	WF-CIV-20 TTL refresh scheduler	E2	Expiring one chunk's findings re-enqueues exactly that chunk at priority 300	Sonnet
E4	E4	Twenty-company pilot and calibration	E2, E3	Citation violations 0; grade-A fill >=60%; cost per company <$1.00; Stu's three-dossier confirmation	Opus

Critical path. A1 -> A2 -> A3 -> C2 -> C3 -> D3 -> D4 -> E1 -> E2 -> E4. Nine hops. Every other work package hangs off it.

Parallelization. B1, B2 and B4 have no dependency on the Phase A queue work beyond A1 and should start immediately — B4 in particular is on nobody's critical path but blocks six jurisdiction datapoints, so starting it late is the most common way this build ends with an empty section 2 of the dossier. Within Phase D, D2, D3 and D5

are mutually independent once C3 exists and are the widest parallel fan in the build; only D4 must wait, because it reads what D3 stores.

9. Execution model for build agents

9.1 Dispatch protocol

One work package, one fresh agent session. Review the gate evidence before dispatching the next. A failed gate returns to the same work package with the failure output attached, never forward to the next. Never span a phase boundary in a single dispatch — the phase gates in 9.3 exist because the failure modes they catch are invisible from inside a single package.

9.2 Briefing template

```
You are building work package <WP-ID> of the CiV Dossier Engine.

ENVIRONMENT
- Supabase project: hoyinfxpotpwkhzsgime (MCP tools available)
- n8n: self-hosted, MCP tools available. Credentials already present:
    Compliance Supabase  Nm1h0bjPqIZc2MAB
    OpenRouter           WkivISO0KPmV0dNq
    CourtListener Token  pCvA69grsS2sgc4P
    Tavily               aGz5l9Pc4hhk9VXd
    Apify                KC4i5wo7SNt8e4XS
    Slack                sykoBGqJPyOBA1Kz
    Airscale             vRVJcn1nRmJtDkLw
- Plan: plans/civ-dossier-engine-implementation.md, Task <N>. Execute it verbatim.

HARD RULES
- P2: never relax the citation_gate constraint. If an insert fails it, fix the caller.
- P4: only civ_project_company() writes companies.civ.
- P5: if your package is an archetype 'sql' chunk, no model call may appear anywhere in it.
- P6: never dispatch a datapoint whose method = 'ask' to a model.
- No Claude, Anthropic or OpenAI model in any runtime path. Kimi K2 and DeepSeek V3 only.

GATE
<paste the Gate cell from the section 8 table>

Run the verification queries from Task <N> and paste their actual output. If the output does
not match what the task says to expect, STOP and report the mismatch. Do not improvise a fix,
do not adjust the expectation, and do not proceed to the next task.
```

9.3 Phase gates

Gate	After phase	Evidence required
G-A	A	Output of the deliberately-failing insert showing `violates check constraint "citation_gate"`
G-B	B	Catalog counts by category, and the archetype distribution query
G-C	C	One company's `civ_chunk_run` row at status `done` with N01–N10 findings listed
G-D	D	`civ_citation_violation` = 0 plus ten hand-verified quotes with their URLs
G-E	E	The four pilot numbers: completion, per-category fill, citation integrity, cost per company

9.4 Model selection rationale

Packages that are fully specified by the plan — DDL application, column additions, catalog loading, a scheduled workflow with fixed nodes — go to Sonnet. Packages with real degrees of freedom go to Opus: the dependency view (A3) because a subtly wrong `NOT EXISTS` produces a queue that looks right and runs chunks out of order; the prompts (B3) because they are the system's only defence against fabrication before the constraint catches it; the runner (C3) because seven archetypes share one control flow; the disqualifier (D1) because name matching is judgement; and every gate-bearing package in D and E.

Escalation rule. Two failed gates on the same work package under Sonnet — re-dispatch to Opus with both failure transcripts attached.

Note that this selection governs the **build agents only**. The runtime contains no Claude model of any kind: collection runs on Kimi K2 and DeepSeek V3 through OpenRouter, per the constraint that Claude stays out of the CiV agent stack.

10. Environment and credentials matrix

Resource	Identifier	Status	Who acts
Supabase project	`hoyinfxpotpwkhzsgime`	Exists, 504 companies loaded	Build agent
n8n Compliance Supabase	`Nm1h0bjPqIZc2MAB`	Exists	Build agent
OpenRouter	`WkivISO0KPmV0dNq`	Exists	Build agent
CourtListener token	`pCvA69grsS2sgc4P`	Exists	Build agent
Tavily search	`aGz5l9Pc4hhk9VXd`	Exists	Build agent
Apify	`KC4i5wo7SNt8e4XS`	Exists	Build agent
Slack	`sykoBGqJPyOBA1Kz`	Exists	Build agent
Airscale (LinkedIn)	`vRVJcn1nRmJtDkLw`	Exists	Build agent
NANPA refresh workflow	`LX7srZ3KlMeQVmYy`	Exists and active	Build agent calls it
`pg_trgm` extension	—	**Must be confirmed enabled** — required by D1 name matching	**User action item**
Slack channels `#civ-ops`, `#civ-dossiers`	—	**Must be created**	**User action item**
Supabase Storage bucket `dossiers`	—	**Must be created**	**User action item**
Kimi K2 / DeepSeek V3 spend limit on OpenRouter	—	**Must be set** before the full run	**User action item**
n8n queue mode (Redis + workers)	—	**Must be configured on Hostinger** — see risk 2	**User action item**
Apify monthly plan	—	**Must be confirmed** sized for ~30,000 page renders per pass	**User action item**
Tavily plan	—	**Must be confirmed** sized for ~11,000 searches per pass	**User action item**

No credential in this table is created by a build agent. Everything marked as a user action item is a secret, a billing decision, or an infrastructure change that only Rad can make.

11. Risk register

#	Risk	Likelihood	Impact	Mitigation
1	A fabricated datapoint reaches a dossier Stu acts on	Medium	Severe — destroys trust in all 134	`citation_gate` CHECK (A1), runner-side enforcement (C3), `civ_citation_violation` at 100% coverage (A4), PC-22 release gate (E2)
2	n8n stays in default single-process mode; a full pass takes a week and risks out-of-memory	High if not addressed	High	Queue mode listed as a user action item in section 10; the queue design already supports multiple workers via `FOR UPDATE SKIP LOCKED` (A2)
3	Concurrent chunks overwrite each other in `companies.civ`	High if P4 is violated	High — silent data loss	Findings moved to their own table with PK `(company_id, datapoint_id)` (A1); `companies.civ` written only by `civ_project_company()` (A4)
4	Name collision disqualifies a real prospect at SCR-01	Medium	High — the prospect is lost silently	Adjudication step using state and DBA (D1); disqualified share must land 20–40% or the package fails its gate; every disqualification stores its reason
5	`civ_state_overlay` never gets seeded; six jurisdiction datapoints stay empty	Medium	Medium	B4 is a named work package with its own gate; PC-20's verification explicitly expects 0 findings until B4 runs
6	Apify returns static HTML; D04 (pre-checked box) and G03 (call-tracking DIDs) become unanswerable	Medium	Medium	D3's gate requires a non-empty `checkboxes` array with `defaultChecked` present in a stored artifact before D4 may start
7	DeepSeek drifts on strict JSON under load	High	Low	One repair retry with the validator error appended (D4); a second failure marks the run `failed` and the reaper retries it
8	Unattended spend runs away	Medium	High	`civ_budget` daily cap checked as the first node of every workflow (C1); `max_tool_calls` per chunk (B2); OpenRouter spend limit as a user action item
9	Stuck claims silently stall the queue	Medium	Medium	30-minute reaper in WF-CIV-00 (C1)
10	The catalog's grade-C datapoints prove uncollectable at scale, and coverage disappoints	Medium	Low	E4's per-category fill query makes it visible at 20 companies rather than 504; grade-A fill >=60% is the gate, grade-C is not gated

12. Assumptions and out of scope

Assumptions. The 134-row catalog supplied on 2026-08-20 is the collectible subset of a larger list; the chunk graph's `produces` arrays reference 160 codes, the extra 26 being 16 ask-only datapoints and the 10 computed `S` scores, so 26 codes in the graph will have no catalog row until that fuller list is loaded — WP B1's gate reports them rather than inventing them. The `civ_chunk_def` dependency graph is assumed correct as authored; this plan does not re-derive it. `companies.industry_type` is assumed to be a usable vertical label — it currently reads 258 solar and 220 HVAC with no insurance rows, so the insurance segment of the stated ICP is not represented in the present 504 and no work package will surface that absence. Public sources are assumed to remain reachable without authentication; a source that begins requiring login becomes an ask-only datapoint rather than a build failure.

Out of scope. No outbound message of any kind is generated — no cold email, no one-pager, no deck. The deliverable is the dossier; a human decides what to send. State-court litigation (N03) is out of scope in v1 because access is per-county and largely paywalled. PC-24, consent change history from Wayback Machine snapshots, is marked optional in the graph and deferred to v2. YouMail data is not integrated; the telephony chunks use NANPA and the public complaint corpora only. Attio and Instantly synchronisation, though credentialed and present in the stack, is a downstream concern once dossiers exist and is deliberately not wired into this engine.

Appendix A — Implementation plan (construction drawings)

What follows is the complete task-level plan. Every task carries its exact DDL or node list, the code to be typed, and a verification query whose expected output is stated. Build agents execute from here; Part I above exists to explain why these instructions are shaped the way they are.

Goal: Run the 30-chunk CiV collection graph over 504 prospect companies in Supabase project `hoyinfxpotpwkhzsgime`, unattended, orchestrated by n8n on Hostinger, using Kimi K2 and DeepSeek V3 through the existing OpenRouter credential, producing one evidence-cited research dossier per company. No Claude anywhere in the runtime path.

Grounded facts (verified against the live systems on 2026-08-20, not assumed):

- `companies` holds **504 rows**; all 504 have `website`, 489 have `phone`; 258 solar, 220 HVAC, 26 other.
- `civ_chunk_def` holds **30 chunks** with `depends_on` / `produces` arrays already populated.
- `civ_artifact` exists and is empty — its comment already states the rule this plan enforces:

"the stored bytes behind every quoted string. No finding may reference text absent from here."

- `civ_state_overlay` exists and is **empty** — the jurisdiction chunk PC-20 cannot produce

anything until REF-02 fills it.

- `companies.civ` and `companies.civ_chunks` exist as `jsonb`, both `{}` for all 504 rows.
- Reference data already loaded: `npanxx_blocks` 204,462 · `fcc_violations` 80,536 ·

`ftc_dnc` 466,196 · `court_cases` 6,071 · `vendors` 14 · `vendor_sellers` 35 · `wholesale_voip_carriers` 14 · `carrier_line_type_rules` 56.

- n8n holds 105 workflows. **None of them implement any CIV chunk.** The `civ_chunk_def.skill`

column points at Claude Code slash commands (`/civ-pc-01-entity`), so the existing design has no n8n execution layer at all.

- n8n credentials that exist and will be used: OpenRouter `WkivISO0KPmV0dNq`,

Compliance Supabase `Nm1h0bjPqIZc2MAB`, CourtListener Token `pCvA69grsS2sgc4P`, Tavily `aGz5l9Pc4hhk9VXd`, Apify `KC4i5wo7SNt8e4XS`, Slack `sykoBGqJPyOBA1Kz`.

The one-sentence architecture: the dependency graph in `civ_chunk_def` is already a correct work-breakdown; this plan adds the three things it lacks — a row-based queue that can be claimed concurrently, a findings table that makes an uncited value impossible to store, and an n8n execution layer that replaces the Claude-skill assumption.

Phase A — Foundation

Task A1 — Findings table and the citation gate

File: apply as Supabase migration `civ_engine_a1_findings`.

The existing design stores collected values in `companies.civ jsonb`. Twenty-four per-company chunks run concurrently against one company; every one of them would read-modify-write that same column. Concurrent `jsonb_set` on one row is a lost-update race — the last writer silently discards the others. Findings move to their own table, one row per company × datapoint, and `companies.civ` becomes a projection written by exactly one chunk (PC-23) at the end.

```
create table civ_datapoint (
  datapoint_id         text primary key,
  category             text not null,
  label                text not null,
  definition           text not null,
  why_tcpa             text not null,
  primary_source       text,
  grade                text not null check (grade in ('A','B','C')),
  publicly_collectible text not null check (publicly_collectible in ('Yes','Partial')),
  collection_route     text,
  public_gives         text,
  public_lacks         text,
  discovery_question   text,
  method               text not null check (method in ('api','web_extract','derived','ask')),
  ttl_days             int  not null default 90,
  requires_render      boolean not null default false,
  chunk_id             text references civ_chunk_def(chunk_id)
);
comment on table civ_datapoint is
  'The catalog. method=ask means never dispatched to an agent - it renders as a discovery-call quest
    ion.';

create table civ_finding (
  company_id         uuid not null references companies(id) on delete cascade,
  datapoint_id       text not null references civ_datapoint(datapoint_id),
  chunk_id           text not null references civ_chunk_def(chunk_id),
  run_id             bigint,
  value_json         jsonb,
  confidence         text check (confidence in ('high','medium','low')),
  method             text not null check (method in ('api','web_extract','derived','ask')),
  artifact_id        bigint references civ_artifact(artifact_id),
  source_url         text,
  evidence_quote     text,
  reason_unavailable text,
  grade              text check (grade in ('A','B','C')),
  collected_at       timestamptz not null default now(),
  expires_at         timestamptz not null,
  primary key (company_id, datapoint_id),

  constraint citation_gate check (
    value_json is null
    or method = 'derived'
    or (artifact_id is not null
        and source_url is not null
        and evidence_quote is not null
        and length(evidence_quote) >= 12)
  )
);
create index civ_finding_expiry_ix on civ_finding (expires_at);
create index civ_finding_chunk_ix  on civ_finding (chunk_id);

comment on constraint citation_gate on civ_finding is
  'A value with no stored artifact and no verbatim quote cannot be inserted. Derived values (PC-21 s
    cores) cite datapoints, not URLs, and are exempt.';
```

Verify:

```
-- must raise: new row for relation "civ_finding" violates check constraint "citation_gate"
insert into civ_finding (company_id, datapoint_id, chunk_id, value_json, method, expires_at)
select id, 'A01', 'PC-01', '"Acme LLC"'::jsonb, 'web_extract', now() + interval '90 days'
from companies limit 1;
```

Expected: `ERROR: new row for relation "civ_finding" violates check constraint "citation_gate"`. If that insert succeeds, the constraint is wrong and the whole evidence guarantee is void — stop.

Commit: `A1: civ_datapoint catalog + civ_finding with citation gate constraint`

Task A2 — Run queue with concurrent claiming

File: migration `civ_engine_a2_queue`.

```
create table civ_chunk_run (
  run_id            bigserial primary key,
  company_id        uuid not null references companies(id) on delete cascade,
  chunk_id          text not null references civ_chunk_def(chunk_id),
  status            text not null default 'pending'
                    check (status in ('pending','claimed','done','failed','dead','skipped')),
  attempts          int  not null default 0,
  max_attempts      int  not null default 3,
  priority          int  not null default 100,
  claimed_at        timestamptz,
  claimed_by        text,
  started_at        timestamptz,
  finished_at       timestamptz,
  model             text,
  prompt_tokens     int,
  completion_tokens int,
  tool_calls        int,
  fetch_count       int,
  cost_usd          numeric(10,4),
  findings_written  int,
  last_error        text,
  created_at        timestamptz not null default now()
);

create unique index civ_chunk_run_active_uk
  on civ_chunk_run (company_id, chunk_id) where status <> 'dead';
create index civ_chunk_run_claim_ix
  on civ_chunk_run (priority, run_id) where status = 'pending';

alter table companies
  add column if not exists civ_status text not null default 'new'
    check (civ_status in ('new','queued','running','disqualified','gated','complete','failed'));
create index companies_civ_status_ix on companies (civ_status);
```

The partial unique index is the idempotency guarantee: a re-fired scheduler cannot enqueue the same company × chunk twice while a live attempt exists.

```
create or replace function civ_claim_chunks(p_worker text, p_limit int default 8)
returns setof civ_chunk_run
language sql as $$
  update civ_chunk_run r
     set status = 'claimed', claimed_at = now(), started_at = now(),
         claimed_by = p_worker, attempts = r.attempts + 1
   where r.run_id in (
     select run_id from civ_chunk_run
      where status = 'pending'
      order by priority, run_id
      limit p_limit
      for update skip locked
   )
  returning r.*;
$$;
```

`FOR UPDATE SKIP LOCKED` is what allows more than one n8n worker to pull from this queue without collision. Without it, adding workers produces duplicate work rather than throughput.

Verify:

```
insert into civ_chunk_run (company_id, chunk_id)
select id, 'SCR-01' from companies limit 3;
select count(*) from civ_claim_chunks('verify-worker', 2);   -- expect 2
select status, count(*) from civ_chunk_run group by 1;        -- expect claimed=2, pending=1
delete from civ_chunk_run where claimed_by = 'verify-worker' or status = 'pending';
```

Commit: `A2: civ_chunk_run queue + civ_claim_chunks with SKIP LOCKED`

Task A3 — Dependency resolution view

File: migration `civ_engine_a3_ready`.

This view is the entire orchestrator. It answers "which company × chunk pairs are runnable right now" directly from the dependency graph already stored in `civ_chunk_def.depends_on`. n8n never encodes the chunk order — it selects from this view.

```
create or replace view civ_ready_chunk as
select c.id            as company_id,
       d.chunk_id,
       d.sort_order,
       case when d.rate_limited then 10 else 100 end as priority
  from companies c
 cross join civ_chunk_def d
 where d.per_company = true
   and d.optional    = false
   and c.civ_status in ('queued','running')
   and not exists (
         select 1 from civ_chunk_run r
          where r.company_id = c.id and r.chunk_id = d.chunk_id and r.status <> 'dead')
   and not exists (
         select 1 from unnest(d.depends_on) dep
          where not exists (
                select 1 from civ_chunk_run r2
                 where r2.company_id = c.id and r2.chunk_id = dep and r2.status = 'done'));
```

Verify:

```
update companies set civ_status = 'queued'
 where id = (select id from companies order by company_name limit 1);
select chunk_id from civ_ready_chunk;   -- expect exactly one row: SCR-01
```

Only `SCR-01` may appear: it is the sole per-company chunk with an empty `depends_on`. If more than one chunk appears, `depends_on` is not being honoured — stop and fix before Phase C.

Commit: `A3: civ_ready_chunk dependency-resolution view`

Task A4 — Budget guard, citation audit view, projection function

File: migration `civ_engine_a4_guards`.

```
create table civ_budget (
  day          date primary key default current_date,
  llm_cost_usd numeric(10,4) not null default 0,
  fetch_count  int           not null default 0,
  llm_cap_usd  numeric(10,2) not null default 20,
  fetch_cap    int           not null default 6000,
  halted       boolean       not null default false
);
insert into civ_budget (day) values (current_date) on conflict do nothing;
```

The citation audit needs no HTTP fetch and no sampling. `civ_artifact` already stores the bytes every quote came from, so verification is a substring test in SQL across 100% of findings:

```
create or replace view civ_citation_violation as
select f.company_id,
       f.datapoint_id,
       f.chunk_id,
       f.source_url,
       left(f.evidence_quote, 80) as quote_head,
       case when a.artifact_id is null then 'artifact_missing'
            else 'quote_not_in_artifact' end as violation
  from civ_finding f
  left join civ_artifact a on a.artifact_id = f.artifact_id
 where f.value_json is not null
   and f.method <> 'derived'
   and (a.artifact_id is null or position(f.evidence_quote in a.body) = 0);
```

Single-writer projection back into `companies.civ`, called once per company by PC-23:

```
create or replace function civ_project_company(p_company uuid)
returns jsonb
language plpgsql as $$
declare v jsonb;
begin
  select coalesce(jsonb_object_agg(f.datapoint_id, jsonb_build_object(
           'v', f.value_json, 'c', f.confidence, 'src', f.source_url,
           'q', f.evidence_quote, 'g', f.grade, 'at', f.collected_at)), '{}'::jsonb)
    into v
    from civ_finding f
   where f.company_id = p_company;

  update companies
     set civ = v, civ_updated_at = now(),
         civ_chunks = coalesce((select jsonb_object_agg(r.chunk_id, r.status)
                                from civ_chunk_run r where r.company_id = p_company), '{}'::jsonb)
   where id = p_company;
  return v;
end $$;
```

Verify:

```
select count(*) from civ_citation_violation;                      -- expect 0 (table empty)
select civ_project_company((select id from companies limit 1));    -- expect {}
select halted, llm_cap_usd from civ_budget where day = current_date;  -- expect false, 20.00
```

Commit: `A4: budget guard, citation-violation view, single-writer projection`

Phase B — Definitions

Task B1 — Load the 134-datapoint catalog

File: `scripts/load_catalog.py` in the CiV ops repo; source CSV `datapoint_catalog_collectible.csv`.

The CSV carries 134 rows across 18 categories (A–R). It is the publicly-collectible subset; the chunk graph's `produces` arrays reference 160 codes, the extra 26 being 16 ask-only datapoints and the 10 computed `S` scores. Rows load with `datapoint_id` assigned as category letter plus a zero-padded ordinal within category, matching the codes already in `civ_chunk_def.produces` (`A01`…`A12`, `B01`…`B10`, and so on).

```
import csv, json, os, re, urllib.request

CSV = os.environ["CATALOG_CSV"]
PGREST = os.environ["SUPABASE_URL"] + "/rest/v1/civ_datapoint"
KEY = os.environ["SUPABASE_SERVICE_KEY"]

# TTL by category: how fast the fact actually changes.
TTL = {"A": 365, "B": 180, "C": 90, "D": 30, "E": 30, "F": 90, "G": 30, "H": 90,
       "I": 60, "J": 90, "K": 180, "L": 180, "M": 14, "N": 7,  "O": 30, "P": 365,
       "Q": 30,  "R": 60}

# Datapoints whose value only exists in a rendered DOM (JavaScript executed).
RENDER = {"D02", "D03", "D04", "D05", "D13", "G03"}

rows, counters = [], {}
for r in csv.DictReader(open(CSV)):
    if not r.get("Datapoint"):
        continue
    letter = r["Category"].strip()[0]
    counters[letter] = counters.get(letter, 0) + 1
    dp_id = f"{letter}{counters[letter]:02d}"
    partial = r["Publicly collectible"].strip() != "Yes"
    lacks = (r.get("What it does not give you") or "").strip()

    rows.append({
        "datapoint_id": dp_id,
        "category": r["Category"].strip(),
        "label": r["Datapoint"].strip(),
        "definition": r["Definition"].strip(),
        "why_tcpa": r["Why it matters (TCPA)"].strip(),
        "primary_source": r["Primary source"].strip(),
        "grade": r["Grade"].strip(),
        "publicly_collectible": "Partial" if partial else "Yes",
        "collection_route": r["Public collection route"].strip(),
        "public_gives": (r.get("What the public web gives you") or "").strip(),
        "public_lacks": lacks,
        # The CSV's "what it does not give you" column IS the discovery-call question.
        "discovery_question": (f"{lacks[0].upper()}{lacks[1:]}?" if partial and lacks else None),
        "method": "ask" if partial and not r["Public collection route"] else
                  ("web_extract" if partial else
                   ("api" if re.search(r"api|registry|dataset|nanpa|courtlistener|rpc",
                                       r["Public collection route"], re.I) else "web_extract")),
        "ttl_days": TTL[letter],
        "requires_render": dp_id in RENDER,
    })

req = urllib.request.Request(
    PGREST, data=json.dumps(rows).encode(), method="POST",
    headers={"apikey": KEY, "Authorization": f"Bearer {KEY}",
             "Content-Type": "application/json",
             "Prefer": "resolution=merge-duplicates"})
print(urllib.request.urlopen(req).status, len(rows))
```

Then bind each datapoint to the chunk that produces it, straight from the existing graph:

```
update civ_datapoint d
   set chunk_id = c.chunk_id
  from (select chunk_id, unnest(produces) as dp from civ_chunk_def) c
 where c.dp = d.datapoint_id;
```

Verify:

```
select count(*) from civ_datapoint;                                  -- expect 134
select count(*) from civ_datapoint where chunk_id is null;           -- expect 0
select count(*) from civ_datapoint where discovery_question is not null;  -- expect 25
select category, count(*) from civ_datapoint group by 1 order by 1;
-- expect A 12, B 10, C 12, D 15, E 8, F 6, G 10, H 9, I 5, J 2,
--        K 1, L 2, M 8, N 10, O 5, P 6, Q 5, R 8
```

Any row with `chunk_id is null` is a datapoint the graph does not produce — report the list rather than inventing a chunk for it.

Commit: `B1: load 134-datapoint catalog, bind to chunk graph, derive discovery questions`

Task B2 — Add the n8n execution layer to civ_chunk_def

File: migration `civ_engine_b2_exec`.

The graph currently describes *what* each chunk produces and *what it depends on*, but its only execution hint is `skill` — a Claude Code slash command. These columns describe how n8n runs it.

```
alter table civ_chunk_def
  add column archetype          text,
  add column model              text,
  add column tool_allowlist     text[],
  add column evidence_kinds     text[],
  add column system_prompt      text,
  add column max_tool_calls     int default 12,
  add column max_evidence_chars int default 60000;

comment on column civ_chunk_def.skill is
  'LEGACY: Claude Code slash command from the pre-n8n design. Not used at runtime. See archetype.';
```

Seven archetypes, assigned so that no model runs where deterministic SQL will do:

```
update civ_chunk_def set archetype = 'sql'      where chunk_id in
  ('REF-03','REF-04','REF-05','PC-04','PC-11','PC-12','PC-20','PC-22');
update civ_chunk_def set archetype = 'api'      where chunk_id in ('SCR-01','PC-18');
update civ_chunk_def set archetype = 'fetch'    where chunk_id in ('PC-05','PC-06','PC-10','PC-24');
update civ_chunk_def set archetype = 'extract'  where chunk_id in ('PC-07','PC-08','PC-09','PC-14');
update civ_chunk_def set archetype = 'research' where chunk_id in
  ('REF-01','REF-02','PC-01','PC-02','PC-03','PC-13','PC-15','PC-16','PC-17','PC-19');
update civ_chunk_def set archetype = 'compute'  where chunk_id = 'PC-21';
update civ_chunk_def set archetype = 'compose'  where chunk_id = 'PC-23';

update civ_chunk_def set model = case archetype
  when 'research' then 'moonshotai/kimi-k2'
  when 'extract'  then 'deepseek/deepseek-chat'
  when 'api'      then 'deepseek/deepseek-chat'
  when 'compose'  then 'deepseek/deepseek-chat'
  else null end;

update civ_chunk_def set tool_allowlist = case archetype
  when 'research' then array['tavily_search','apify_scrape','pg_read']
  when 'fetch'    then array['apify_render','apify_scrape','pg_read']
  when 'extract'  then array[]::text[]
  when 'api'      then array['http_request','pg_read']
  else array[]::text[] end;
```

Only 10 of 30 chunks carry an agentic research model; 8 run as pure SQL with no model at all.

Verify:

```
select archetype, count(*), count(model) as with_model
  from civ_chunk_def group by 1 order by 1;
-- expect: api 2/2, compose 1/1, compute 1/0, extract 4/4,
--         fetch 4/0, research 10/10, sql 8/0
select count(*) from civ_chunk_def where archetype is null;  -- expect 0
```

Commit: `B2: archetype, model and tool allowlist columns on civ_chunk_def`

Task B3 — Extraction prompts

File: migration `civ_engine_b3_prompts`.

One prompt per chunk, stored as data so retuning never touches n8n. The shared preamble is the part that prevents fabrication and must not be softened.

```
update civ_chunk_def set system_prompt = $prompt$
You are a research extraction agent for Compliance IV. You receive evidence collected from
public sources about one company and a list of datapoints to extract from that evidence.

ABSOLUTE RULES
1. Extract only what the supplied evidence literally supports. You may not use background
   knowledge about this company, its industry, or its competitors.
2. Every non-null value MUST carry artifact_id, source_url and evidence_quote. The
   evidence_quote must be a VERBATIM substring of the supplied evidence, at least 12
   characters, that a person could locate on that page.
3. If the evidence does not support a datapoint, set value to null and write
   reason_unavailable. Returning null is a correct and expected outcome. A wrong value is
   far worse than a missing one, because a human will act on it.
4. Never infer, estimate, approximate or round unless the datapoint definition says to.
5. Reproduce the value in the type the datapoint declares. Do not reformat quotes.

Return JSON matching the supplied schema, with one object per datapoint id, including the
ids you could not find.
$prompt$
where archetype in ('extract','research','api','compose');
```

Per-chunk task text is appended to that preamble at dispatch time. Three examples; the same pattern covers the remaining chunks and each is written in its own work package.

```
update civ_chunk_def set system_prompt = system_prompt || $t$

CHUNK TASK — PEWC element extraction (PC-07).
You are reading consent disclosures captured next to phone-number fields. For each of the
six PEWC elements of 47 CFR 64.1200(f)(9), decide whether the element is PRESENT, ABSENT or
AMBIGUOUS, quoting the exact clause you relied on. D09 (not a condition of purchase) is the
element most often missing — do not infer it from adjacent marketing language; it must be
stated. Where several brand domains disagree, report the weakest surface and name the brand.
$t$ where chunk_id = 'PC-07';

update civ_chunk_def set system_prompt = system_prompt || $t$

CHUNK TASK — Messaging programme extraction (PC-08).
You are reading SMS terms and booking-flow disclosures. Extract the disclosed message
frequency verbatim (E03) rather than normalising it. STOP and HELP keyword disclosure must
be quoted from the terms page itself, not inferred from a generic footer. If a dedicated SMS
terms page does not exist in the evidence, set E01 false with a reason and leave E02-E08
null - absence of the page is itself the finding.
$t$ where chunk_id = 'PC-08';

update civ_chunk_def set system_prompt = system_prompt || $t$

CHUNK TASK — Suppression and revocation extraction (PC-09).
You are reading the published do-not-call policy. 47 CFR 64.1200(d)(3) allows a maximum of
ten business days to honour an opt-out; extract the stated window (I09) verbatim and do not
convert units. For I08, a policy that names one exclusive revocation channel is a finding,
not a gap - quote the limiting sentence. You cannot observe whether the policy is honoured
in practice; leave those datapoints null with reason_unavailable set to 'internal_practice'.
$t$ where chunk_id = 'PC-09';
```

Verify:

```
select chunk_id, length(system_prompt) from civ_chunk_def
 where archetype in ('extract','research','api','compose') order by chunk_id;
-- every row non-null; PC-07, PC-08 and PC-09 longer than the shared preamble
select count(*) from civ_chunk_def
 where archetype in ('extract','research','api','compose') and system_prompt is null;  -- expect 0
```

Commit: `B3: shared anti-fabrication preamble + per-chunk task prompts`

Task B4 — Seed civ_state_overlay

File: migration `civ_engine_b4_overlay` plus REF-02 first run.

`civ_state_overlay` is empty, and PC-20 produces six datapoints (P01–P06) entirely from it. Until it is seeded, every company's jurisdiction section is blank. The table's own comment sets the rule: *"Primary-source statutory text only. verified=false blocks the citation gate."*

Seed the states in the current footprint — the 504 companies' operating states — beginning with the mini-TCPA states that carry a private right of action, since those are what stack damages: Florida (FTSA), Washington (CEMA), Oklahoma (OTSA), Maryland (MTCPA), New York, Michigan, Texas, Louisiana, Mississippi, Arkansas, Connecticut, and Rhode Island.

Every row requires `citation`, `canonical_url` and a `verification_string` that appears verbatim at that URL, and stays `verified = false` until REF-01 confirms it against the primary source.

Verify:

```
select count(*) from civ_state_overlay;                      -- expect >= 12
select count(*) from civ_state_overlay where verified;       -- expect >= 12 after REF-01
select state from civ_state_overlay
 where has_mini_tcpa and private_right_of_action order by state;
select count(*) from civ_state_overlay
 where verification_string is null or canonical_url is null;  -- expect 0
```

Commit: `B4: seed civ_state_overlay for mini-TCPA footprint states`

Phase C — Core orchestration

Task C1 — WF-CIV-00 Guard and Reaper

n8n workflow. Schedule `*/5 * * * *`. Credentials: Compliance Supabase `Nm1h0bjPqIZc2MAB`, Slack `sykoBGqJPyOBA1Kz`.

Nodes in order:

1. **Schedule Trigger** — every 5 minutes.
2. **Postgres · ensure budget row** — `insert into civ_budget (day) values (current_date) on conflict do nothing;`
3. **Postgres · roll up spend**

```
update civ_budget b set
  llm_cost_usd = coalesce((select sum(cost_usd) from civ_chunk_run
                           where finished_at::date = current_date), 0),
  fetch_count  = coalesce((select sum(fetch_count) from civ_chunk_run
                           where finished_at::date = current_date), 0)
 where b.day = current_date
returning *;
```

4. **Postgres · set halt flag**

```
update civ_budget set halted = (llm_cost_usd >= llm_cap_usd or fetch_count >= fetch_cap)
 where day = current_date returning halted, llm_cost_usd, llm_cap_usd;
```

5. **Postgres · reap stuck claims**

```
update civ_chunk_run
   set status = 'pending', claimed_at = null, claimed_by = null
 where status = 'claimed' and claimed_at < now() - interval '30 minutes'
returning run_id;
```

6. **Postgres · kill exhausted**

```
  update civ_chunk_run set status = 'dead'
   where status = 'failed' and attempts >= max_attempts returning run_id, chunk_id;
```

7. **IF** `{{ $json.halted }}` is true -> **Slack** to `#civ-ops`:

```
Budget halt: ${{ $json.llm_cost_usd }} of ${{ $json.llm_cap_usd }} spent today. Engine paused.
```

Verify: set the cap to zero and confirm the halt fires, then restore it.

```
update civ_budget set llm_cap_usd = 0 where day = current_date;
-- execute WF-CIV-00 manually; expect halted = true and one Slack message
select halted from civ_budget where day = current_date;   -- expect true
update civ_budget set llm_cap_usd = 20, halted = false where day = current_date;
```

Commit: `C1: WF-CIV-00 budget guard and stuck-claim reaper`

Task C2 — WF-CIV-01 Scheduler

n8n workflow. Schedule `*/15 * * * *`. Credential: Compliance Supabase `Nm1h0bjPqIZc2MAB`.

1. **Schedule Trigger** — every 15 minutes.
2. **Postgres · budget check** — `select halted from civ_budget where day = current_date;`
3. **IF** halted -> **NoOp** (stop).
4. **Postgres · admit companies** — keep a bounded working set so the queue never runs away:

```
  update companies set civ_status = 'queued'
   where id in (
     select id from companies
      where civ_status = 'new' and website is not null and website <> ''
      order by company_name
      limit greatest(0, 40 - (select count(distinct company_id) from civ_chunk_run
                              where status in ('pending','claimed')))
   ) returning id;
```

5. **Postgres · enqueue ready chunks** — the dependency view decides, not n8n:

```
  insert into civ_chunk_run (company_id, chunk_id, priority)
  select company_id, chunk_id, priority from civ_ready_chunk
  on conflict do nothing
  returning run_id, company_id, chunk_id;
```

6. **Postgres · promote to running** —

```
update companies set civ_status = 'running' where civ_status = 'queued' and id in (select
company_id from civ_chunk_run where status <> 'dead');
```

Verify:

```
update companies set civ_status = 'new';
delete from civ_chunk_run;
-- execute WF-CIV-01 manually
select count(*) from civ_chunk_run;                  -- expect 40 (one SCR-01 per admitted company)
select distinct chunk_id from civ_chunk_run;         -- expect exactly SCR-01
```

Only SCR-01 may be enqueued on the first pass. Anything else means `depends_on` is being ignored.

Commit: `C2: WF-CIV-01 scheduler driven by civ_ready_chunk`

Task C3 — WF-CIV-10 Chunk Runner

n8n workflow. Schedule `*/5 * * * *`. Credentials: Compliance Supabase `Nm1h0bjPqIZc2MAB`, OpenRouter `WkivISO0KPmV0dNq`, Tavily `aGz5l9Pc4hhk9VXd`, Apify `KC4i5wo7SNt8e4XS`, CourtListener `pCvA69grsS2sgc4P`.

This is the only workflow that runs chunk logic. Archetype dispatch replaces the 30 separate Claude skills the original design assumed.

1. **Schedule Trigger** — every 5 minutes.
2. **Postgres · budget check** -> **IF** halted -> stop.
3. **Postgres · claim** — `select * from civ_claim_chunks('{{ $execution.id }}', 8);`
4. **IF** no rows -> stop.
5. **Split In Batches** — batch size 1. Everything below runs per claimed chunk.
6. **Postgres · load chunk spec**

```
select d.chunk_id, d.archetype, d.model, d.tool_allowlist, d.system_prompt,
       d.max_tool_calls, d.max_evidence_chars, d.evidence_kinds,
       c.id as company_id, c.company_name, c.website, c.state, c.industry_type,
       (select jsonb_agg(jsonb_build_object(
          'id', dp.datapoint_id, 'label', dp.label, 'definition', dp.definition,
          'grade', dp.grade, 'hint', dp.collection_route) order by dp.datapoint_id)
          from civ_datapoint dp
         where dp.chunk_id = d.chunk_id and dp.method <> 'ask') as datapoints
  from civ_chunk_def d
  join companies c on c.id = $1
 where d.chunk_id = $2;
```

The `method <> 'ask'` filter is load-bearing: the 25 partially-collectible datapoints are never shown to a model, so it is never placed in a position to invent them.

7. **Postgres · load evidence** (extract and compose archetypes only)

```
select artifact_id, kind, url, left(body, 20000) as body
  from civ_artifact
 where company_id = $1 and kind = any($2)
 order by fetched_at desc limit 12;
```

8. **Switch** on `archetype` — seven branches:

- **sql** -> Postgres node executing the chunk's query (Phase D supplies each).
- **api** -> HTTP Request -> small DeepSeek adjudication call.
- **fetch** -> HTTP Request to Apify -> writes `civ_artifact`, writes no findings.
- **extract** -> HTTP Request to OpenRouter with the evidence bundle.
- **research** -> AI Agent node, tools Tavily + Apify, model from the chunk row.
- **compute** -> Postgres scoring query.
- **compose** -> template Code node + one DeepSeek summary call.

9. **Code · validate and enforce** — schema check, then the belt-and-braces citation rule:

```
const out = [];
for (const dp of $json.datapoints ?? []) {
  const hasEvidence = dp.artifact_id && dp.source_url &&
                      dp.evidence_quote && dp.evidence_quote.length >= 12;
  if (dp.value !== null && dp.value !== undefined &&
      !hasEvidence && dp.method !== 'derived') {
    dp.value = null;
    dp.confidence = null;
    dp.reason_unavailable = 'model_returned_value_without_citation';
  }
  out.push(dp);
}
return out.map(j => ({ json: j }));
```

The database constraint from Task A1 would reject these anyway; catching them here converts a failed insert into a clean `reason_unavailable` and keeps the chunk's other findings.

10. **Postgres · write findings**

```
insert into civ_finding (company_id, datapoint_id, chunk_id, run_id, value_json,
                         confidence, method, artifact_id, source_url, evidence_quote,
                         reason_unavailable, grade, expires_at)
select $1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, dp.grade,
       now() + (dp.ttl_days || ' days')::interval
  from civ_datapoint dp where dp.datapoint_id = $2
on conflict (company_id, datapoint_id) do update set
  value_json = excluded.value_json, confidence = excluded.confidence,
  artifact_id = excluded.artifact_id, source_url = excluded.source_url,
  evidence_quote = excluded.evidence_quote,
  reason_unavailable = excluded.reason_unavailable,
  collected_at = now(), expires_at = excluded.expires_at;
```

11. **Postgres · close the run** — status `done`, `finished_at`, token counts, `cost_usd`, `findings_written`.
12. **Error Trigger branch** — `update civ_chunk_run set status='failed', last_error=$1 where run_id=$2;`

WF-CIV-00 promotes it to `dead` after three attempts.

Verify: run one company end-to-end through SCR-01 only.

```
select r.chunk_id, r.status, r.model, r.cost_usd, r.findings_written
  from civ_chunk_run r order by r.run_id desc limit 5;   -- expect status done
select datapoint_id, value_json, left(evidence_quote,50), source_url
  from civ_finding where chunk_id = 'SCR-01';            -- expect N01..N10 rows
select count(*) from civ_citation_violation;             -- expect 0
```

A non-zero `civ_citation_violation` count after the first real chunk means the runner is writing quotes that are not in the stored artifact — stop and fix before enabling any other chunk.

Commit: `C3: WF-CIV-10 chunk runner with seven-archetype dispatch`

Phase D — Chunk implementations

Each work package implements one archetype's chunk bodies inside the WF-CIV-10 Switch built in C3. They are independent of each other and can be built in parallel once C3 exists.

Task D1 — SCR-01 litigation disqualifier (archetype `api`)

The ICP is prevention-only: CiV does not take clients already sued. The catalog says so directly — datapoint N02 is annotated *"ICP disqualifier and risk signal"*. This is the cheapest chunk and it removes the most work, so it runs first for every company and gates everything else.

1. **Postgres · docket match against data already held** (no API call, no model):

```
   select cc.*, similarity(lower(cc.defendant), lower($2)) as name_sim
     from court_cases cc
    where lower(cc.defendant) % lower($2)
    order by name_sim desc limit 10;
```

Requires `create extension if not exists pg_trgm;`.

2. **HTTP Request · CourtListener** for anything not already in `court_cases`, credential `pCvA69grsS2sgc4P`, endpoint `/api/rest/v4/search/?type=r&q=<name>&suitNature=485`. Throttle to 5 requests per minute — `rate_limited` is already true on this chunk, so the scheduler gives it priority 10 and the runner processes it in small claims.
3. **DeepSeek adjudication** — company names collide. "Solar Solutions LLC" in Arizona is not the one in Florida. Supply the docket defendant string, the company legal name, DBA and state; ask for a boolean plus a one-line reason. Roughly 2,000 tokens.
4. **Write** N01–N10 findings, storing the docket text in `civ_artifact` first so the quotes verify.
5. **Switch on the verdict:**

- match -> `update companies set civ_status='disqualified' where id=$1;` and mark every other pending chunk for that company `skipped`.
- no match -> leave `civ_status='running'`; the scheduler picks up PC-01 on its next pass.

Verify:

```
select civ_status, count(*) from companies group by 1;
select c.company_name, f.value_json
  from civ_finding f join companies c on c.id = f.company_id
 where f.datapoint_id = 'N02' and f.value_json::text <> 'null' limit 10;
-- Manually open three matched dockets and confirm the defendant really is this company.
select count(*) from civ_chunk_run where status = 'skipped';
```

The disqualified share should land between 20% and 40%. Outside that band the name matching is wrong in one direction or the other — review before proceeding, because a false positive silently removes a real prospect and nothing downstream will surface it.

Commit: `D1: SCR-01 litigation disqualifier with name adjudication`

Task D2 — SQL chunks (archetype `sql`, 8 chunks)

No model runs in any of these. Each is a Postgres node in the WF-CIV-10 Switch.

- **PC-04 crawl plan** — allocate a page budget per brand domain from `civ_finding` A10; write the plan to `civ_artifact` with `kind='crawl_plan'`. Cap: 40 pages per company, 12 per brand.
- **PC-11 number classification** — join harvested numbers to `npanxx_blocks`, `carrier_line_type_rules` and `wholesale_voip_carriers`; call the existing `classify_tns` RPC. Produces G05–G09 and S07.
- **PC-12 complaint corpus join** — join attributed numbers to `fcc_violations` (80,536 rows) and `ftc_dnc` (466,196 rows). Produces M01–M05. The artifact is the matched complaint rows serialised into `civ_artifact` so the quotes verify.
- **PC-20 jurisdiction and vendor overlay** — join operating states (B04) to `civ_state_overlay` and detected vendors to `vendors`. Produces P01–P06 and Q05.
- **PC-22 assertion gate** — the release gate:

```
  select count(*) as violations from civ_citation_violation where company_id = $1;
```

Non-zero blocks PC-23. Also rejects any finding whose text asserts a legal conclusion; CiV's standing rule is that an allegation is never described as a finding.

- **REF-03 NANPA load** — already implemented as the active workflow *NANPA Monthly Refresh* (`LX7srZ3KlMeQVmYy`). Call it rather than rebuilding; mark the chunk done from its result.
- **REF-04 vendor signatures** / **REF-05 vendor risk graph** — recompute `vendors.case_count`, `seller_count`, `settled_count`, `last_seen` from `court_cases`. Produces Q01–Q04.

Verify:

```
select chunk_id, count(*) runs, count(*) filter (where status='done') done
  from civ_chunk_run where chunk_id in
   ('PC-04','PC-11','PC-12','PC-20','PC-22','REF-03','REF-04','REF-05')
 group by 1 order by 1;
select count(*) from civ_finding where chunk_id = 'PC-12';   -- > 0 for companies with numbers
select count(*) from civ_finding where chunk_id = 'PC-20';   -- 0 until B4 seeds the overlay
select model from civ_chunk_run
 where chunk_id in ('PC-04','PC-11','PC-12','PC-20','PC-22') and model is not null;  -- expect 0 row
    s
```

That last check is the point of the archetype split: if any SQL chunk recorded a model, an LLM ran where a join belonged.

Commit: `D2: eight SQL chunk bodies, no model in the path`

Task D3 — Fetch chunks (archetype `fetch`, 3 mandatory)

These write `civ_artifact` and no findings. Apify credential `KC4i5wo7SNt8e4XS`.

- **PC-05 consent document acquisition** — fetch `/privacy`, `/terms`, `/sms-terms`, `/messaging-terms`, `/do-not-call`, `/dnc-policy` and sitemap-discovered equivalents per brand domain. Produces E01, I01, I02 as existence booleans; stores each page body.
- **PC-06 rendered form probe** — **must execute JavaScript.** D04 is whether a consent checkbox is pre-checked, which is only readable as the `defaultChecked` property in a live DOM, and G03 is the call-tracking DID pool, which only appears after the tracking script runs. Use an Apify actor running real Chrome, and render each form page three times to surface the DID rotation.

```
  // Apify page function — returned to n8n and stored as the artifact body.
  const forms = [...document.querySelectorAll('form')].map(f => ({
    action: f.action,
    hasPhone: !!f.querySelector('input[type=tel], input[name*=phone i]'),
    checkboxes: [...f.querySelectorAll('input[type=checkbox]')].map(c => ({
      name: c.name, defaultChecked: c.defaultChecked, checked: c.checked,
      label: (c.closest('label') || {}).innerText || ''
    })),
    consentText: f.innerText.slice(0, 4000),
    privacyLinkAdjacent: !!f.querySelector('a[href*=privacy]')
  }));
  const tels = [...document.body.innerHTML.matchAll(/\(?\d{3}\)?[-.\s]?\d{3}[-.\s]?\d{4}/g)]
                 .map(m => m[0]);
  return { url: window.location.href, forms, tels: [...new Set(tels)] };
```

- **PC-10 number inventory harvest** — crawl every page of every brand domain for telephone numbers. The catalog is explicit that main lines alone do not match complaint data (G02: *"Main lines alone do not match complaint data — this is the fix"*), so this must be site-wide, not the contact page.

Every artifact row is written with its `sha256`; on a refresh run an unchanged hash skips the dependent extract chunk entirely.

Verify:

```
select kind, count(*), avg(length(body))::int avg_bytes
  from civ_artifact group by 1 order by 1;
select count(*) from civ_artifact where sha256 is null;   -- expect 0
-- PC-06 must have produced defaultChecked data:
select body::jsonb -> 'forms' -> 0 -> 'checkboxes'
  from civ_artifact where kind = 'form_probe' limit 1;
```

If `checkboxes` is empty across every company, the render is returning static HTML and D04 will be unanswerable — fix the actor before running PC-07.

Commit: `D3: PC-05/PC-06/PC-10 acquisition into civ_artifact, real Chrome for form probe`

Task D4 — Extract chunks (archetype `extract`, 4 chunks)

DeepSeek V3 via OpenRouter `WkivISO0KPmV0dNq`, `response_format` set to a strict JSON schema. These read `civ_artifact` and never touch the network.

- **PC-07 PEWC elements** — D06–D12, D14 from the consent surfaces.
- **PC-08 messaging programme** — C02, C06, E02–E08, E10, J03 from SMS terms.
- **PC-09 suppression and revocation** — I06–I10, J01, H08, K01–K06 from the DNC policy.
- **PC-14 technology fingerprint** — H01, H04, H05, H07, H09–H11, C11, C12 from stored page source.

Response schema, sent with every call:

```
{
  "type": "object",
  "required": ["datapoints"],
  "additionalProperties": false,
  "properties": {
    "datapoints": {
      "type": "array",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "required": ["datapoint_id","value","confidence","artifact_id",
                     "source_url","evidence_quote","reason_unavailable"],
        "properties": {
          "datapoint_id":       {"type": "string"},
          "value":              {"type": ["string","number","boolean","array","null"]},
          "confidence":         {"type": ["string","null"], "enum": ["high","medium","low",null]},
          "artifact_id":        {"type": ["integer","null"]},
          "source_url":         {"type": ["string","null"]},
          "evidence_quote":     {"type": ["string","null"]},
          "reason_unavailable": {"type": ["string","null"]}
        }
      }
    }
  }
}
```

DeepSeek drifts on strict JSON under load. The runner performs exactly one repair retry, resending with the validator error appended; a second failure marks the run `failed`. Expect the repair path to fire on roughly 3–8% of calls — that is normal, not a defect.

Verify:

```
select chunk_id,
       count(*) filter (where value_json is not null) as filled,
       count(*) filter (where value_json is null)     as nulls
  from civ_finding where chunk_id in ('PC-07','PC-08','PC-09','PC-14')
 group by 1 order by 1;
select count(*) from civ_citation_violation;   -- expect 0
```

Then verify ten quotes by hand: open the `source_url` and search for the `evidence_quote` string. Fewer than nine of ten found means the prompt is broken — fix before scaling past the pilot.

Commit: `D4: PC-07/PC-08/PC-09/PC-14 extraction against stored artifacts`

Task D5 — Research chunks (archetype `research`, 8 per-company)

Kimi K2 via OpenRouter, tools Tavily search `aGz5l9Pc4hhk9VXd` and Apify scrape `KC4i5wo7SNt8e4XS`, capped at `max_tool_calls`. Every page the agent reads is written to `civ_artifact` before any finding referencing it can be stored — this is what makes the citation gate hold for agentic chunks as well as extraction ones.

- **PC-01 entity and ownership** — A01–A08, A12. Secretary of State, site footer, sponsor pages.
- **PC-02 brand and domain discovery** — A09–A11. Certificate-transparency SANs via `crt.sh`,

sitemaps, brand pages. This chunk determines the crawl surface for everything downstream, so its recall matters more than any other research chunk.

- **PC-03 scale and footprint** — B01–B10, C09. Operating states here (B04) feed PC-20's

jurisdiction overlay, so a missed state silently drops a mini-TCPA exposure.

- **PC-13 complaint narratives** — M06–M08, G11. BBB profiles, 800notes and equivalent boards.
- **PC-15 hiring signals** — R07, C01, H02, H03, H06, L03, L05. Careers pages and job boards.

Job posts are how the dialer platform (H02) and the BPO relationship (H06) become public.

- **PC-16 vendor case studies** — C03, C04, C07, C10, C14, J05. Vendors publish their clients'

cadences; this is where a multi-day automated sequence becomes provable.

- **PC-17 lead sources** — F01–F03, F08, F09. Aggregator directories, Google Local Services Ads.
- **PC-19 commercial and people** — R01–R06, R08, L01, L02. Leadership pages and LinkedIn.

Route LinkedIn through the existing Airscale credential `vRVJcn1nRmJtDkLw` rather than scraping it directly.

Verify:

```
select chunk_id, count(*) runs,
       round(avg(cost_usd)::numeric, 4) avg_cost,
       round(avg(extract(epoch from finished_at - started_at))::numeric, 0) avg_secs,
       round(avg(tool_calls)::numeric, 1) avg_tools
  from civ_chunk_run where chunk_id like 'PC-%' and model is not null
 group by 1 order by 1;
select count(*) from civ_citation_violation;   -- expect 0
select count(*) from civ_finding f
  join civ_datapoint d using (datapoint_id)
 where d.category like 'A.%' and f.value_json is not null;
```

Any chunk averaging its `max_tool_calls` ceiling is being truncated mid-research — raise the cap for that chunk or narrow its datapoint list.

Commit: `D5: eight agentic research chunks on Kimi K2 with artifact-first writes`

Phase E — Scoring, gate, dossier, pilot

Task E1 — PC-21 scoring (archetype `compute`)

No model. Produces S01–S10 from findings already collected, with `method='derived'` so the citation gate exempts them — but each must record its inputs:

```
insert into civ_finding (company_id, datapoint_id, chunk_id, value_json, method, grade, expires_at)
select $1, 'S01', 'PC-21',
       jsonb_build_object(
         'score', <expression>,
         'derived_from', array['C01','C02','D09','G06','M01','I01']),
       'derived', 'A', now() + interval '30 days';
```

Weights come from `scoring_weights` (currently version 1, `is_active = true`) rather than being hard-coded, so Stu can retune without a workflow edit.

Verify:

```
select f.value_json ->> 'score' as score, f.value_json -> 'derived_from'
  from civ_finding f where f.datapoint_id = 'S01' order by 1 desc limit 10;
select count(*) from civ_finding
 where datapoint_id like 'S%' and value_json -> 'derived_from' is null;  -- expect 0
```

Commit: `E1: PC-21 exposure scoring from scoring_weights`

Task E2 — PC-22 gate and PC-23 dossier

PC-22 blocks release. A company with any row in `civ_citation_violation` cannot render.

PC-23 composes the dossier from a **deterministic template**, not an agent — identical structure across 504 companies is what makes them scannable, and a template cannot invent. One DeepSeek call produces a three-sentence "why this company, why now" summary, constrained to reference only datapoints that carry evidence.

Dossier sections, in order:

1. Identity and brands (A) with the claimed-versus-verified brand gap called out.
2. Scale and footprint (B), operating states mapped to mini-TCPA statutes by name.
3. Contact channels and cadence (C, J) — what they actually run.
4. Consent posture (D, E) — the six PEWC elements as a present/absent/ambiguous table, each with its verbatim quote. This is the heart of the dossier.
5. Suppression and revocation (I, K).
6. Telephony (G) and complaint corpus (M), joined on attributed numbers.
7. Litigation and enforcement history (N, O) — allegations described as allegations.
8. Vendors and lead sources (F, H, Q).
9. Exposure score (S) with its component breakdown.
10. People and entry points (L, R).
11. **Discovery-call questions** — auto-generated from every datapoint where `method = 'ask'` or `value_json is null`, rendered from `civ_datapoint.discovery_question`.

Then `select civ_project_company($1);` — the single writer — and `update companies set civ_status = 'complete';`.

Write the file to Supabase Storage as `dossiers/{industry_type}/{company_name}.md` and post the link to Slack `#civ-dossiers` with company name, exposure score and the count of unanswered questions.

Verify:

```
select count(*) from civ_citation_violation;                      -- expect 0
select civ_status, count(*) from companies group by 1;
select id, civ_updated_at, jsonb_object_keys_count(civ) from companies
 where civ_status = 'complete' limit 5;
```

Then read three dossiers cold. The test is whether Stu can write an opening line from each without opening another tab. Three failures means the composition template is wrong, not the collection.

Commit: `E2: PC-22 citation gate and PC-23 deterministic dossier composition`

Task E3 — Refresh scheduler

n8n workflow WF-CIV-20. Schedule `0 3 * * *`.

```
insert into civ_chunk_run (company_id, chunk_id, priority)
select f.company_id, f.chunk_id, 300
  from civ_finding f
 where f.expires_at < now()
 group by f.company_id, f.chunk_id
having count(*) >= 3
on conflict do nothing;
```

TTLs are set per category in B1: litigation 7 days, complaints 14, consent surfaces 30, telephony 30, entity 365. Priority 300 puts refresh work behind all new-company work. This is the mechanism that turns a one-time 504-company pass into the continuous Early Warning System — the same runner, no new code.

Verify:

```
update civ_finding set expires_at = now() - interval '1 day'
 where company_id = (select id from companies where civ_status='complete' limit 1)
   and chunk_id = 'PC-07';
-- execute WF-CIV-20
select chunk_id, priority, status from civ_chunk_run
 where priority = 300 order by run_id desc limit 5;   -- expect PC-07 pending at 300
```

Commit: `E3: WF-CIV-20 TTL-driven refresh scheduler`

Task E4 — Pilot and calibration

Run 20 companies end to end: 10 solar, 10 HVAC, spread across the seat range and at least six states including Florida and Washington (both carry a private right of action, so the jurisdiction overlay actually gets exercised).

```
update companies set civ_status = 'new'
 where id in (
   select id from companies
    where industry_type ilike '%solar%' order by random() limit 10)
    or id in (
   select id from companies
    where industry_type ilike '%hvac%' order by random() limit 10);
```

Verify — the four numbers that decide whether to run all 504:

```
-- 1. Completion: how many of the 20 reached a dossier
select civ_status, count(*) from companies
 where civ_status <> 'new' group by 1;

-- 2. Coverage: fill rate per category, the honest measure of catalog realism
select d.category,
       count(*) filter (where f.value_json is not null) as filled,
       count(*)                                         as attempted,
       round(100.0 * count(*) filter (where f.value_json is not null) / count(*), 1) as pct
  from civ_finding f join civ_datapoint d using (datapoint_id)
 group by 1 order by 1;

-- 3. Citation integrity: must be zero
select count(*) from civ_citation_violation;

-- 4. Unit economics
select round(sum(cost_usd), 2) as total_usd,
       round(sum(cost_usd) / count(distinct company_id), 3) as usd_per_company,
       round(avg(extract(epoch from finished_at - started_at))::numeric, 0) as avg_chunk_secs
  from civ_chunk_run where status = 'done';
```

Gate to the full run: citation violations exactly 0, at least 60% fill on grade-A datapoints, cost per company under $1.00, and Stu confirming he can write an opener from three dossiers read cold.

Commit: `E4: pilot results and calibration`

Coverage map — catalog to chunk to work package

Catalog categories	Datapoints	Chunks	Work package
N. Litigation	10	SCR-01	D1
A. Entity & identity	12	PC-01, PC-02	D5
B. Scale & footprint	10	PC-03	D5
D. Consent capture	15	PC-06 (acquire), PC-07 (extract)	D3, D4
E. SMS program	8	PC-05 (acquire), PC-08 (extract)	D3, D4
I. DNC & suppression	5	PC-05, PC-09	D3, D4
K. Recordkeeping	1	PC-09	D4
J. Timing & cadence	2	PC-08, PC-09, PC-16	D4, D5
C. Contact channels	12	PC-03, PC-06, PC-08, PC-14, PC-15, PC-16	D3, D4, D5
G. Telephony & numbers	10	PC-10 (harvest), PC-11 (classify)	D3, D2
M. Complaints	8	PC-12 (join), PC-13 (narratives)	D2, D5
H. Technology stack	9	PC-14, PC-15	D4, D5
F. Lead sources	6	PC-17	D5
O. Enforcement	5	PC-18	D1
L. Governance	2	PC-15, PC-19	D5
R. Commercial & outreach	8	PC-19	D5
P. Jurisdiction	6	PC-20 (needs B4 overlay)	D2, B4
Q. Vendor risk	5	PC-20, REF-05	D2
S. Exposure scores (derived)	10	PC-21	E1
Release gate	—	PC-22	E2
Dossier	—	PC-23	E2

All 134 catalog datapoints are claimed by exactly one producing chunk, and every chunk is claimed by exactly one work package.